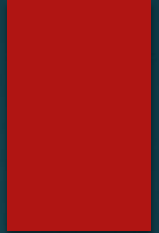


Link List

DATA STRUCTURES

The List Data Structure



- ▶ The list is among the most generic of data structures.

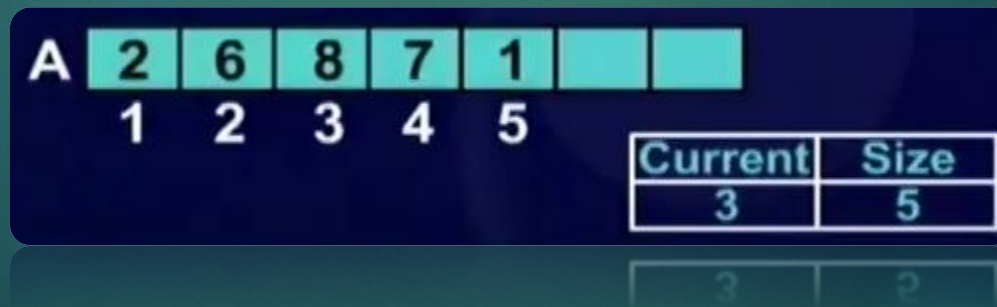
- ▶ Real life:
 - a) Shopping list
 - b) Groceries list
 - c) List of people to invite to dinner
 - d) List of presents to get

Lists

- ▶ A list is collection of items that are all of the same type (grocery items, integers, names)
- ▶ The items, or elements of the list, are stored in some particular order
- ▶ It is possible to insert new elements into various positions in the list and remove any element of the list.

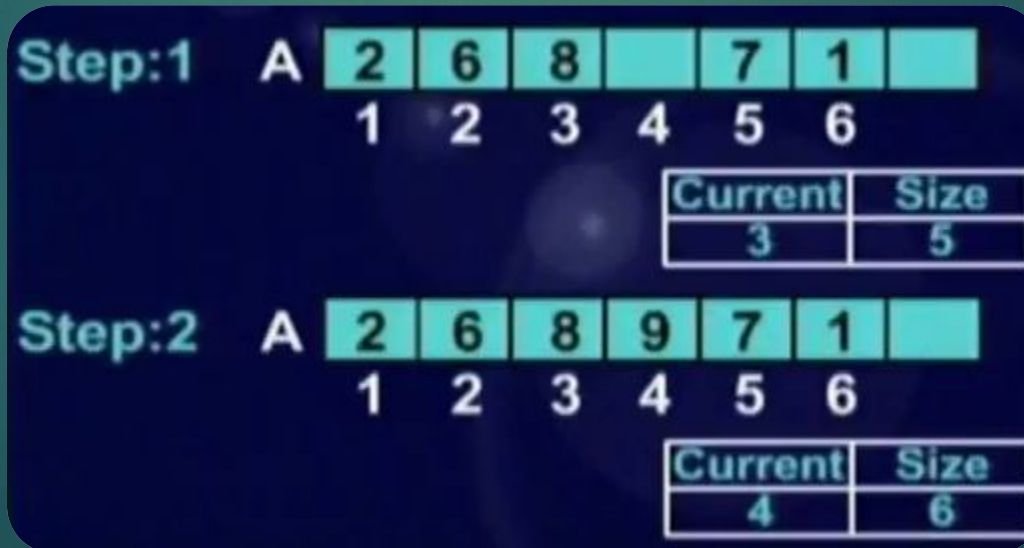
Implementing Lists

- ▶ We have designed the interface for the list;
- ▶ How to implement that interface.
- ▶ Implement list using an array:
- ▶ For example, the list of integers (2,6,8,7,1) could be represented as:



Implement list using an array

- ▶ **add Method**
- ▶ **add(9)**; current position is 3. The new list would thus be: (2,6,8,9,7,1)



Implement list using an array

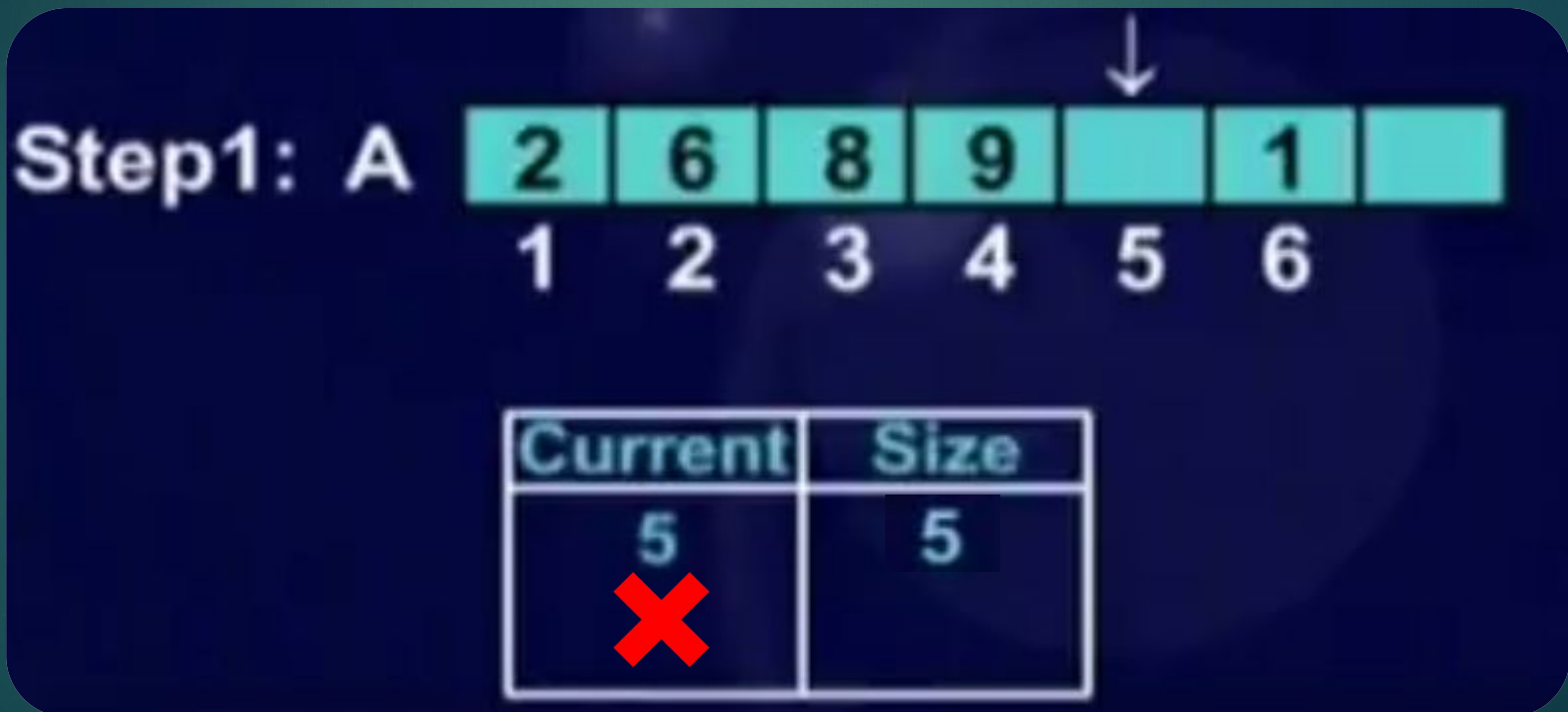
► Next()

A	2	6	8	9	7	1	
	1	2	3	4	5	6	

Current	Size
 5	6

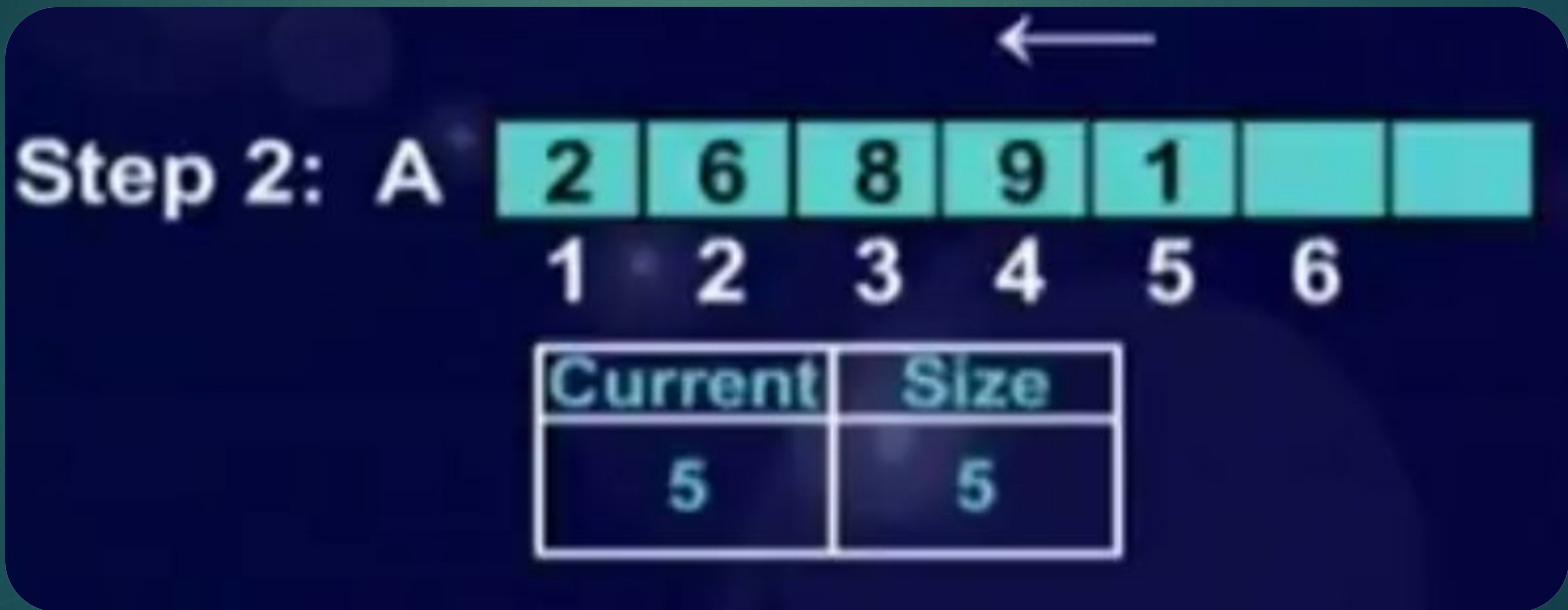
Implement list using an array

- **Remove()**: removes the element at the current index



Implement list using an array

► Remove():

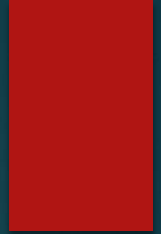


Implement list using an array

```
int find (int x){
    int j ;
    for (j = 1; j < size + 1; j++ )
        {if (A[j] == x ) break;}

    if ( j < size + 1) // x is found
        {current = j ; //current points to the position
                                     where x found
        return 1 ; // return true
        }
    return 0 ; //return false, x is not found
}
```

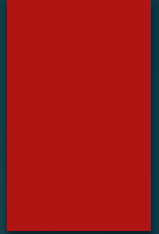
Implement list using an array



- ▶ `get()` □ `return A[current];`
- ▶ `update(X)` □ `A[current]=X;`
- ▶ `length()` □ `return size;`
- ▶ `back()` □ `current--;`
- ▶ `start()` □ `current=1;`
- ▶ `end()` □ `current=size;`

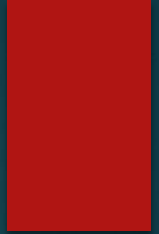
▶ Note: These operation are one step $O(1)$

Analysis of Array Lists



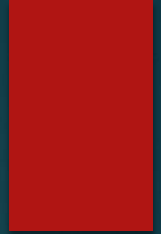
- ▶ add
 - ▶ Have to move every element to make space for new element
 - ▶ Worst case: insert at beginning
- ▶ remove
 - ▶ Worst case: remove at the beginning
- ▶ find
 - ▶ Worst case: may have to search the entire array

Limitations in Array list



- ▶ If we need more elements to store in the array.
 - ▶ it is not possible to increase or decrease it during the execution of the program.
- ▶ Suppose we have used the dynamic memory allocation and created an array of 100 elements with the use of *new* operator.
 - ▶ Need of 200 elements
 - ▶ First copy its elements to the new array
 - ▶ not an easy task.

List Using *Linked Memory*



- ▶ To avoid such problems, there is need of using linked memory in which the various cells of memory, are not located continuously.
- ▶ Each cell of the memory not only contains the value of the element but also the information where the next element of the list is residing in the memory.
- ▶ In this one element is located at some memory location while the other element can be located somewhere far from it in the memory.

Linked List

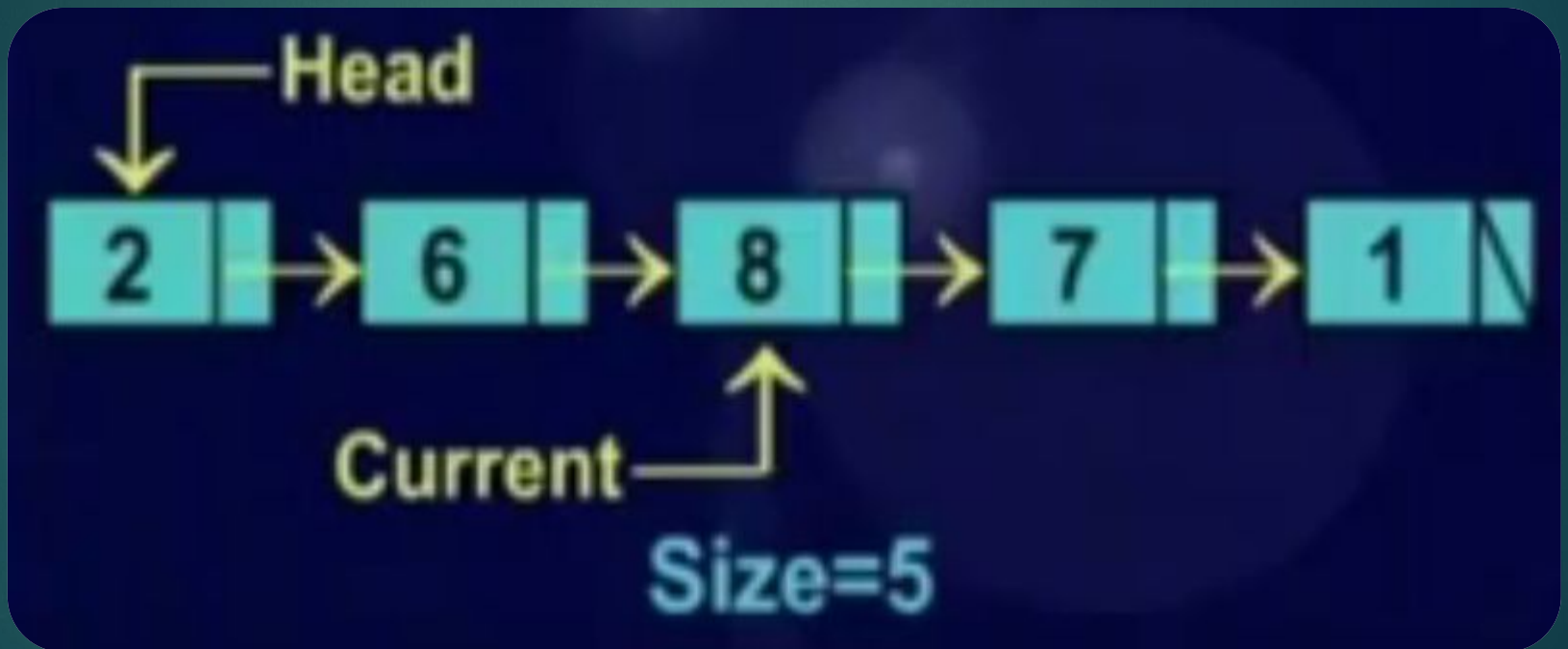
- ▶ Create a structure called a **Node**.



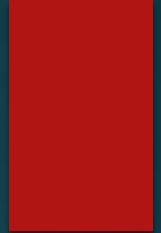
- ▶ The **object** field will hold the actual list element
- ▶ The **next** field in the structure will hold the starting location of the next node.
- ▶ Chain the nodes together to form a **linked** list

Linked List

- ▶ Picture of our list (2,6,7,8,1) stored as a linked list:

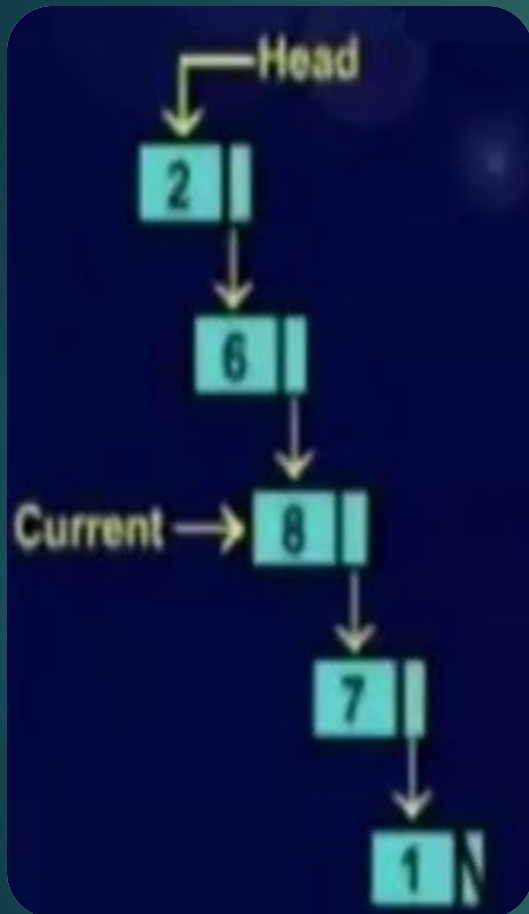


Linked List



- ▶ **Note Some feature of the list:**
 - ▶ Need a **head** to point to the first node of the list. Otherwise we won't know where the start of the list is.
 - ▶ The **current** here is a pointer, not an index.
 - ▶ The next field in the last node points to **nothings**. We will place the memory address NULL which is guaranteed to be inaccessible.

Actual picture In memory



	1051	6
	1052	1063
Current □	1053	1063
	1054	2
	1055	1051
	1056	
	1057	7
	1058	1060
	1059	
	1060	1
	1061	0
Head □	1062	1054
	1063	8
	1064	1057
	1065	

Linked List Operations

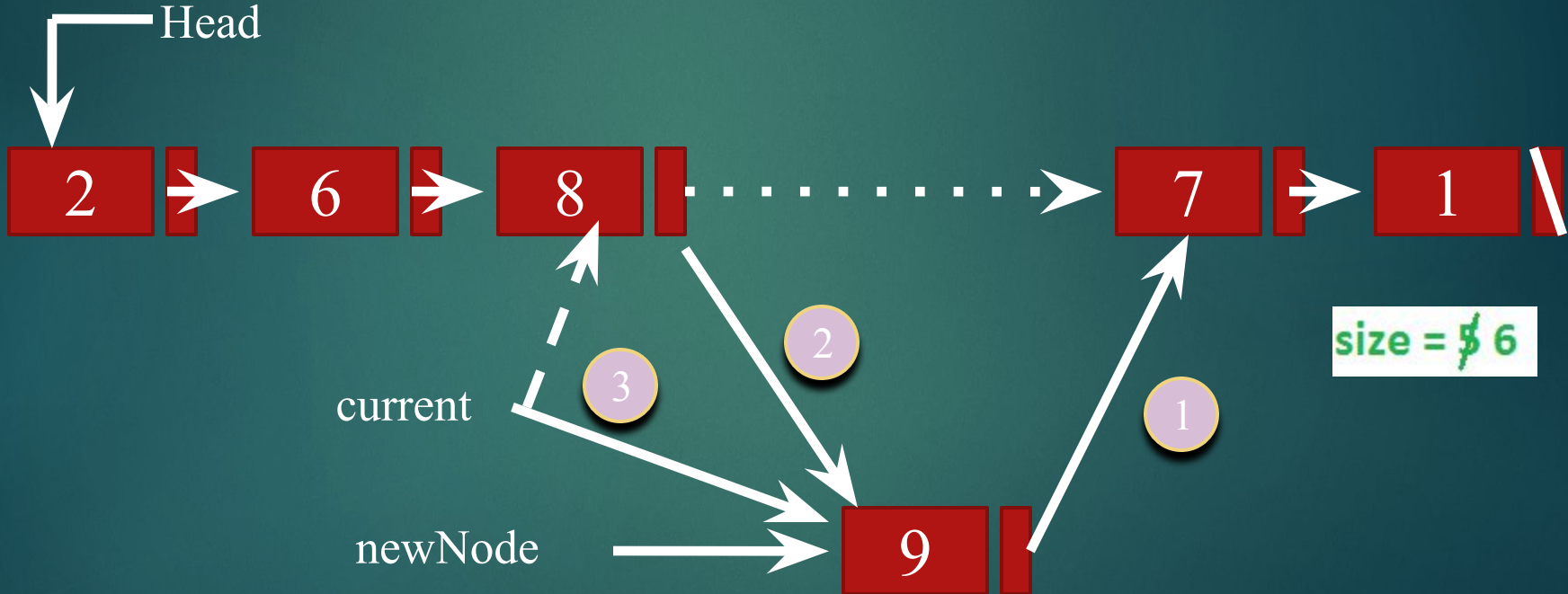
- ▶ `add(9)`: Create a new node in memory to hold '9'

```
Node* newNode = new Node(9);
```



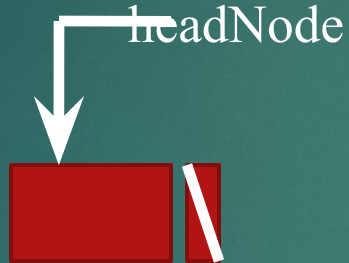
Linked List Operations

- ▶ Link the new node into the list



Building a Link list step-by-step View

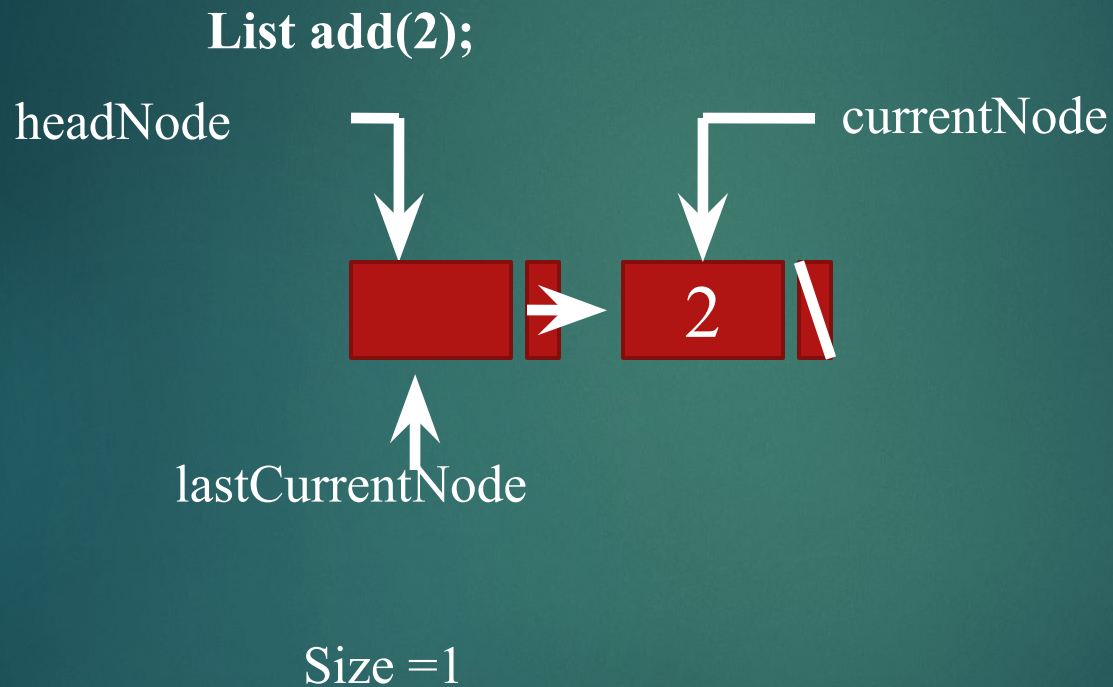
List list;



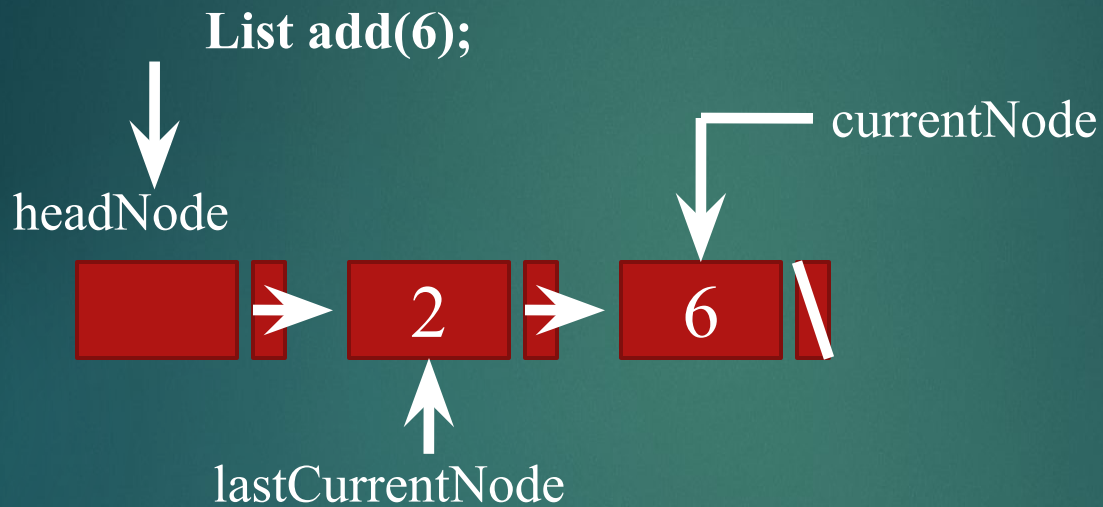
currentNode=NULL
lastCurrentNode=NULL

Size =0

Building a Link list step-by-step View

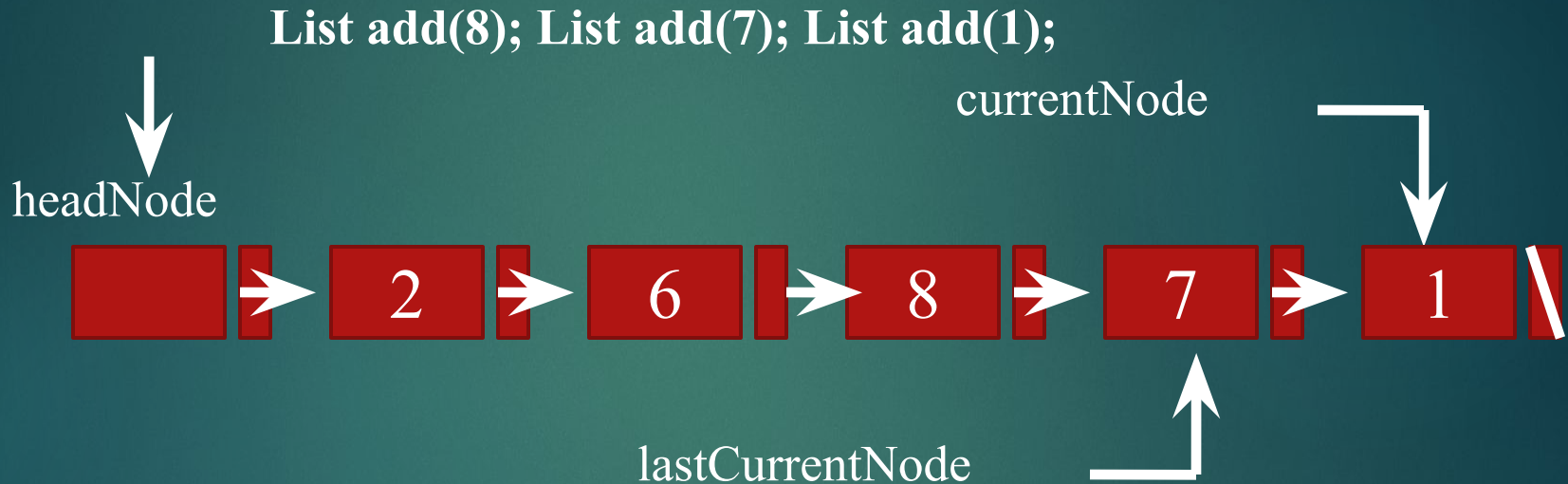


Building a Link list step-by-step View



Size =2

Building a Link list step-by-step View



Size =5

Remove():

